

TP1 - Sistemas Operativos

1. ¿Cuáles son las dos funciones principales de un Sistema Operativo?

El concepto de un sistema operativo puede entenderse desde dos perspectivas principales: de arriba hacia abajo y de abajo hacia arriba. En la **vista de arriba hacia abajo**, el sistema operativo se concibe como un proveedor de abstracciones para los programas de aplicación, facilitando su interacción con el hardware de manera que estos programas puedan funcionar sin preocuparse por los detalles de bajo nivel del hardware. Es decir que desde este punto actúa como intermediario entre un usuario de una computadora y el hardware de la computadora.

Por otro lado, la **vista de abajo hacia arriba** presenta al sistema operativo como un gestor que se encarga de administrar todos los componentes de un sistema complejo. Estos componentes incluyen procesadores, memorias, dispositivos de entrada/salida (E/S), entre otros. La tarea principal del sistema operativo, desde esta perspectiva, es garantizar una asignación ordenada y controlada de estos recursos entre los diversos programas que los solicitan. Es decir, es un programa que gestiona el hardware de la computadora con los recursos con los que cuenta. Por así decirlo, es como decir que se mete en el barro y arregla los problemas. Desde el otro lado, trata de hacerlo más lindo.

2. Cite varios SO que conozca. ¿Puede clasificarlos de alguna manera? ¿Cuáles son los diferentes criterios que utilizaría para la clasificación de los SO?

- Windows
- Linux
- MacOS

En base a mi conocimiento puedo clasificarlos en primer lugar por qué tan comerciales son. Windows actualmente predomina el mercado, sin embargo, he escuchado que Linux es mucho más seguro por lo que sería otra clasificación. Los diferentes criterios que utilizaría sería en primera parte analizar su facilidad de uso, lo que consume si estamos hablando de un so en una computadora móvil, el tiempo de respuesta si estamos jugando a algún juego, o chateando.

También se pueden clasificar de acuerdo al entorno de su implementación, es decir, si es un sistema operativo embebido, de servidor, para uso personal, para sistemas en tiempo real, etc.

Además, se puede clasificar de acuerdo a su implementación y a la cantidad de funciones que se le da al kernel (microkernel, monolítico, en capas, etc.).

3. En la actividad que realizamos en clase The Operating System Zoo, analizamos nueve tipos diferentes de sistemas operativos. Proporcione una lista de aplicaciones para cada uno de estos sistemas (una por tipo de sistema operativo). Esta en modern-operating-systems 1.4

NO LA HICIMOS EN CLASE - COMPLETAR LUEGO (?)

4. ¿Puede utilizar su computadora personal sin sistema operativo? Justifique

La respuesta sencilla es que SI, sin embargo no tendría mucho sentido y sería muy complicado el poder hacer algo útil sin el mismo. Sin SO la computadora no va a saber como utilizar sus componentes de hardware como la RAM, CPU, HDD/SSD, etc. Tendrías que hacer manualmente tus propios programas, e interactuar directamente con el hardware.

5. Cite algún ejemplo de una computadora que no utilice SO. ¿De qué tipo de máquina se trata?

Algunas computadoras hoy en día utilizan sistemas embebidos como una cafetera automática la cual hace uso de microcontroladores (pequeños sistemas de computación integrados en dispositivos electrónicos), y otros tipos de computadoras más viejas como la conocida ENIAC, el Colossus y todas aquellas predecesoras de la primera generación. También se encuentran los circuitos integrados como los dispositivos de hardware con ASIC, los cuales están diseñados para realizar tareas muy específicas y no requieren un sistema operativo.

6. Los sistemas operativos. ¿son dependientes de la arquitectura de la computadora?

Si que son dependientes, por un lado debido al set de instrucciones con la que trabaja la arquitectura de la computadora teniendo que el sistema operativo debe estar escrito para entender y utilizar estas instrucciones. Los sistemas operativos pueden estar diseñados para funcionar en distintas arquitecturas de hardware específica tales como x86 donde predomina Linux, otras como ARM, MIPS, RISC-V (Apple utiliza esta para sus últimos sistemas operativos). Una de las cosas más importantes en los sistemas operativos, es el manejo de interrupciones. Estas interrupciones son una parte importante a su vez de la arquitectura con la que se manejan así que también afirma esta dependencia.

¿Por qué las aplicaciones son dependientes del sistema operativo?

Las aplicaciones compiladas en un sistema operativo no son ejecutables en otros, debido a diferencias en las llamadas al sistema, el formato binario, y los conjuntos de instrucciones de los CPU. Se mencionan tres métodos para hacer que una aplicación funcione en múltiples sistemas operativos: usar lenguajes interpretados como Python, utilizar máquinas virtuales como Java, o portar el código a cada sistema mediante APIs estándar como POSIX. Sin embargo, a pesar de estos enfoques, la movilidad de aplicaciones entre diferentes plataformas sigue siendo difícil debido a variaciones en APIs, formatos binarios, y ABI (Interfaces de Binarios de Aplicaciones). Estos factores hacen que desarrollar aplicaciones multiplataforma sea un reto considerable.

7. ¿De qué manera el usuario utiliza, directa o indirectamente el sistema operativo?

De manera directa, el usuario lo utiliza mediante la interfaz de usuario la cual a su vez te permite de manera visual, gestionar archivos (crear, modificar, copiar, eliminar) o ejecutando directamente las aplicaciones. alguna de estas aplicaciones puede ser incluso la terminal, lo cual por un lado podríamos decir que es la forma más directa en la que nos comunicamos con el sistema operativo.

Y desde el punto de vista de manera indirecta, es cuando el sistema gestiona los recursos. Por ejemplo, cuando estamos haciendo uso de varias aplicaciones a la vez, el sistema operativo es el encargado de dividir las tareas en cuanto a tiempo y espacio. Respecto al tiempo implica que los recursos se compartan por turnos. Por ejemplo, en un sistema con una sola CPU, el sistema operativo asigna la CPU a un programa durante un tiempo determinado, luego a otro, y así sucesivamente. Por el lado del espacio, se refiere a la división de recursos entre varios usuarios o programas de manera simultánea. Un ejemplo de esto es la memoria principal, que se divide para que varios programas puedan residir en ella al mismo tiempo, optimizando el uso de los recursos disponibles. El sistema operativo también administra la asignación de espacio en disco, asegurando que múltiples usuarios puedan almacenar sus archivos en un mismo disco de manera organizada. Otro más, es cómo maneja la interacción entre los dispositivos de E/S basándose en las interrupciones.

8. a) Cite ejemplos donde el Hardware (Hw) influyó en el diseño de los SO

Las veces en los que el hw influyó en los diseños de los sistemas operativos fue con los procesadores multi-core, permitiendo hacer varias tareas a la vez llevando a que los so se vuelvan más complejos ya que la planificación de tareas y optimización de recursos se agradan. Otro ejemplo es que con la llegada de diferentes tipos de dispositivos periféricos como HDD, SSD, Impresoras, provocó que se tengan que crear los controladores que manejan estos dispositivos para permitir interactuar con el hardware. También el cambio de arquitectura de 32 a 64 bits tuvo impacto en el diseño de los Sistemas Operativos.

Completar aca con lo de los celulares moviles, que la profe decia lo de la abateria

b) ejemplos donde los SO influyeron en el diseño de nuevo Hw.
HAcer

9. La construcción de sistemas operativos tienen varios objetivos de diseño, por ejemplo, utilización de recursos, puntualidad, robustez, etc. Brinde un ejemplo de dos objetivos de diseño que pueden contradecirse.

La puntualidad y la utilización de los recursos (timeliness & resource utilization) pueden contradecirse. La gestión eficaz de los recursos (sin colisiones ni interrupciones) puede dar lugar a que los procesos tengan tiempos de espera más prolongados. Supongamos que queremos optimizar la utilización de recursos, entonces todas las tareas que tengan menor prioridad deberán esperar a que los recursos se liberen, lo que podría aumentar la latencia y afectar la puntualidad.

10. Se ha insistido en la necesidad de que un sistema operativo haga uso eficiente de los recursos. ¿Cuándo es apropiado que un sistema operativo deje de lado este principio y consuma recursos? ¿Por qué estos sistemas operativos no son realmente inútiles?

Answer:

Single-user systems should maximize use of the system for the user. A GUI might “waste” CPU cycles, but it optimizes the user’s interaction with the system.

Antes de responder hay que hacer un panorama general. Con recursos nos referimos a la capacidad de la CPU, RAM, del ancho de banda de la red, de las E/S y la de almacenamiento. Entonces utilizar los recursos de manera eficiente significa utilizar la capacidad al máximo, y no “desperdiciar” recursos, aunque puede resultar contradictorio si producto de esto no hay recursos disponibles cuando los necesitamos.

Entonces, podríamos dejar de lado este principio si por ejemplo llenamos un disco al 100%, lo que provocaría que el sistema operativo falle. No se podría usar espacio para archivos temporales, ni escribir en registros, incluso ni para la recolección de basura continua que utilizan los SSD que necesitan de espacio libre para esto. En pocas palabras, si no existe suficiente espacio libre, el rendimiento se degrada. Y así podemos decir lo mismo con otros recursos, donde su uso aumenta y disminuye.

Resumiendo, en periodos de alto uso de los recursos, no es ineficiente no hacer uso de todos ellos; por lo que no son inútiles estos sistemas operativos ya que si el uso de recursos aumenta demasiado hay que asegurarse de que haya capacidad de reserva para manejar una carga alta brevemente. El objetivo es no fallar durante esos periodos de alta carga.

11. Considerando las diferentes definiciones (funcionalidades/objetivos) de sistemas operativos. Considere si el sistema operativo debe incluir aplicaciones tales como navegadores (browsers), programas de correo electrónico. Discuta pros y contras sobre la inclusión de este tipo de aplicaciones.

Por un lado, los pros de contar con browsers en un sistema operativo es que al momento de instalar un nuevo sistema operativo en tu computadora, ya tienes la capacidad de navegar por la red, o de usar alguna otra aplicación. Además si no te gusta el que viene por default, contás con un lugar más accesible y rápido para descargar uno de tu gusto. De otra forma tendrías que instalar mediante algún tipo de hardware externo.

Por el otro, las contras que tiene que ya traiga aplicaciones es que te estaría ocupando espacio extra con algo que quizás no quieras tener o usar. Esta carga en la memoria, podría producir aumento en el uso de recursos y disminuir la performance. Un dato histórico extra, es que a Microsoft lo denunciaron por competencia desleal al incluir el browser Internet Explorer en su sistema operativo.

An argument in favor of including popular applications in the operating system is that if the application is embedded within the operating system, it is likely to be better able to take advantage of features in the kernel and therefore have performance advantages over an application that runs outside of the kernel. Arguments against embedding applications within the operating system typically dominate, however: (1) the applications are applications—not part of an operating system, (2) any performance benefits of running within the kernel are offset by security vulnerabilities, and (3) inclusion of applications leads to a bloated operating system.

12. Es posible escribir un SO en un lenguaje de alto nivel, que ventajas y desventajas tendría.

Si, es posible, de hecho Android tiene parte de su kernel implementado en C, y muchos frameworks de aplicaciones están escritos en Java.

Ventajas:

El código es más compacto, se escribe y entiende más fácil, por lo que también resulta más fácil de testear. Además, los nuevos compiladores optimizantes ayudan a hacer el código assembler más optimizado que incluso algunos programadores humanos. Por último, es más fácil de portear.

Desventajas:

Mayor uso de recursos y menos eficiente. Sin embargo, esto se ve mitigado por los compiladores optimizantes y procesadores modernos en pipeline. Lo que más afecta negativamente al rendimiento del SO es el uso de algoritmos y estructuras de datos ineficientes.

13. ¿Qué es el núcleo (kernel) de un SO?

Esencialmente, el kernel es el principal software del sistema, el corazón del SO, que está siempre ejecutándose en la computadora. Dentro de lo que sería el SO, incluye las funciones esenciales, como el manejo de interrupciones y traps, y el CPU scheduling. En base a como esté ordenada la estructura del SO (micro kernel, monolítico, en capas) se le dan o relegan funciones. Por ejemplo, en un monolítico estaría incluida la función de I/O (mediante los drivers), mientras que en un sistema microkernel esto podría verse en un programa del sistema externo al kernel.

14. Defina las propiedades esenciales de los siguientes tipos de sistemas operativos: a) batch, b) interactivo, c) tiempo compartido, d) tiempo real, e) distribuido.

- a) Los sistemas en batch, principalmente se basan en el **procesamiento de masivas cantidades de programas**, con **mucho I/O** (cantidad de discos y datos). En particular, se procesan programas y conjuntos de programas rutinarios, **sin la**

presencia de usuarios (es decir, input predeterminado). Ejemplo: procesamiento de reclamos en una agencia de seguros, o reportes de ventas en una cadena de tiendas.

- b) Los SO interactivos son los clásicos, que esperan al input del usuario para hacer sus cálculos. A diferencia del batch, acá la respuesta del SO a los inputs del usuario es la más instantánea posible, ya que se basa en espera los inputs del mismo. (completar)
- c) Los SO de tiempo compartido se basan en la **conexión de varios usuarios** realizando cada uno sus tareas, y a partir de eso el **CPU va siendo alojado** a cada una de esas tareas. Supongamos que hay 20 usuarios están logueados, pero 17 de ellos están pensando en hacer otra cosa no relacionada a usar el sistema, entonces la CPU estará asignada solamente a los 3 trabajos que desean el servicio
- d) Los SO de tiempo real tienen al **tiempo como parámetro clave**. Es decir, se basan en realizar acciones dentro de un parámetro de tiempo establecido. Dentro de esto, podemos encontrar dos tipos:
 - **SO de tiempo real crucial**: Deben garantizar que la acción ocurra en el preciso momento que debe ocurrir. Por ejemplo, en la construcción de un auto, el brazo robot debe realizar su acción en un instante preciso, sino el auto puede romperse. Incluso peores como el sistema operativo de una planta nuclear, el cuál errarle en aunque sea algo mínimo puede causar una catástrofe.
 - **SO de tiempo real blando**: Si bien también se necesita que ocurran acciones en momentos precisos, si se llega a errar no pasa nada serio. Ejemplo: sistemas de audio o smartphones.

e) **revisar**

A distributed operating system, in contrast, is one that appears to its users as a traditional uniprocessor system, even though it is actually composed of multiple processors. The users should not be aware of where their programs are being run or where their files are located; that should all be handled automatically and efficiently by the operating system.

True distributed operating systems require more than just adding a little code to a uniprocessor operating system, because distributed and centralized systems

differ in certain critical ways. Distributed systems, for example, often allow applications to run on several processors at the same time, thus requiring more complex processor scheduling algorithms in order to optimize the amount of parallelism.

15. Enumere y explique las características a tener en cuenta en multiprogramación. ¿Qué relación hay entre multiprogramación y tiempo compartido? ¿Existen diferencias con el tiempo compartido?

La multiprogramación consiste en tener varios programas en memoria, de tal manera que cuando algún proceso está esperando I/O, otro proceso toma su lugar y se ejecuta en el CPU, a fin de no perder tiempo. Algunas características son:

- La memoria se divide en varios programas, un programa en ejecución en CPU se denomina proceso.
- Se requiere de hardware especial que evite que un programa se mezcle o interactúe con otro, a fin de preservar seguridad.

A partir del concepto de multiprogramación, surgió el concepto tiempo compartido, en el cual varios usuarios se conectan a una computadora, y la misma se encarga de repartir el tiempo del CPU en estos distintos usuarios según su necesidad. Por ejemplo, si un usuario está realizando un proceso y se va, el PC carga el programa de otra persona en el CPU, y así repartir bien el cómputo.

Si bien en sí son conceptos casi iguales, la multiprogramación hace referencia a un único usuario, y los programas se cargan en CPU cuando un proceso está esperando para así **no perder tiempo de cómputo**, mientras que en tiempo compartido pueden haber varios usuarios conectados haciendo uso, pero se reparte **un poco a cada uno para que sea equitativo**. Después, puede pasar que haga procesos de batch de fondo mientras no haya necesidad de usuarios con programas menores. Esto aseguró que el CPU esté siempre siendo usado. Igualmente, en ambos hay planificación de CPU.

16. En entornos de multiprogramación y tiempo compartido, los usuarios comparten el sistema simultáneamente. Esta situación puede resultar en varios problemas de seguridad. ¿Cuáles son estos problemas? ¿Es posible garantizar el mismo grado de seguridad en una máquina de tiempo compartido como se tiene en una máquina dedicada?

a. What are two such problems?

Denial Of Service (DOS) Attack

One user can stop other users from doing any operations performed on the system.

Integrity of Information

One user can corrupt the confidential information of other users.

Privacy Problem

One user can access the private information of other users secretly.

b. Can we ensure the same degree of security in a time-shared a machine as in a dedicated machine?

- In a dedicated machine the same degree of security as in a time-shared machine is achievable only if when operating system avoids any type of information sharing among the consumers for any purposes read, write who shares the dedicated single computer system.
- Along with this, there is no guarantee regarding bugs in any applications or software, which can also lead to system security vulnerability.
- Hence, these scenarios indicate that there is no total accurate way that can prevent complete data sharing, proper and equal assignment of system assets.

17. ¿Bajo qué circunstancias sería mejor para un usuario utilizar un sistema de tiempo compartido en lugar de un PC o una estación de trabajo monousuario?

Antiguamente, las PCs de una persona eran costosas, por lo que el acceder a los sistemas de tiempo compartido brindaba a muchas personas la oportunidad de acceder a una computadora para realizar sus tareas, cosa que de otra manera no podrían realizar.

Aparte de eso, un sistema de tiempo compartido podría servir si dentro de una empresa, por ejemplo, se tiene una base de datos grande y todos los empleados necesitan acceder a consultar cosas. Entonces, en vez de que cada empleado tenga todos los datos en su PC, o que haya solo una PC en la empresa que todos consultan, se podría tener un sistema de tiempo compartido que sea accedido por cada empleado desde su PC personal.

18. Describa las diferencias entre multiprocesamiento simétrico y asimétrico. ¿Cuáles son las ventajas y desventajas de un sistema con multiprocesadores?

En el multiprocesamiento simétrico todos los procesadores se les dan las mismas tareas, es decir, se los trata de la misma manera. Hay una memoria compartida por todos los procesadores a la que acceden, independientemente de que cada procesador tenga su memoria caché local.

Por el contrario, en el multiprocesamiento asimétrico a cada procesador se les asigna una tarea (por ejemplo, uno que se dedique al sistema operativo y otro al I/O). En este sistema, hay un procesador central que asigna las tareas, y procesadores secundarios que las cumplen.

Un sistema con multiprocesadores tiene la ventaja de que, al tener varios procesadores interconectados, se pueden distribuir las tareas, por lo que los procesos se completan más rápido. No obstante, un problema que podría surgir este tipo de procesadores es que se asigne mucha carga a uno mientras que otro está desocupado. Esto se podría mitigar haciendo estructuras de datos compartidas y un buen diseño de sistema.

Además, si se empiezan a agregar muchos procesadores el bus del sistema se podría ver sobrecargado, por lo que una buena alternativa es que cada CPU tenga su propia memoria, y que a su vez están interconectados (NUMA).

Otro problema podría ser la complejidad adicional que tener más procesadores conlleva, es decir, cómo se planifican las tareas, etc.

19. ¿Cuál es la diferencia entre una interrupción y un trap? ¿Cuál es el uso de cada una?

Una interrupción es, justamente, una interrupción que realiza un dispositivo de un controlador de I/O al CPU, informando que realizó una operación de I/O. Cuando ocurre una interrupción, se guarda el estado de lo que se está haciendo, y después se envía a un receptor de interrupciones. Ahí el CPU determina de qué tipo se trata (de acuerdo a un número que se ve en un vector de interrupciones), y posteriormente se la da al manejador de interrupciones. Entonces, se atiende la interrupción y después se sigue con lo que se había pausado antes. Estas interrupciones sirven para realizar servicios de I/O.

Un trap es similar a una interrupción, en el sentido en que se interrumpe al SO y se tienen que manejar cosas, sin embargo esta interrupción es realizada por el **software**, ya sea por

un error (división por cero o acceso de memoria inválido), o por una solicitud de un programa de usuario mediante una **llamada al sistema** (razón por la cual los traps son útiles).

20. ¿Por qué es necesaria la operación en “modo dual”?

Para evitar que programas maliciosos tomen el control del SO y empiezan a cambiar cosas del sistema mismo, y de otras aplicaciones, además de no dañar los dispositivos. Esto se evita con este bit de kernel, que simboliza que el sistema operativo tomó el control del sistema, y puede realizar acciones como gestionar I/O e interrupciones o traps.

21. ¿Cómo determina el sistema en que modo se encuentra?

En el hardware hay un bit (bit de modo) que guarda el modo actual del sistema. Hay algunos sistemas que inclusive tienen más de 2 bits, teniendo 4 por ejemplo.

22. ¿Por qué es necesario que el hardware ofrezca mecanismos de protección de I/O, de memoria y de la CPU? Explique cómo se realiza dicha protección.

Capítulo 1.6 del Silberschatz

23. Cuáles de las siguientes instrucciones deben ser privilegiadas:

a) Fijar (set) el valor de un timer

Recontra, el timer evita que los programas caigan en ciclos infinitos por ejemplo, si se modifica puede irse todo al carajo.

b) Leer el reloj

No, no tiene nada de malo.

c) Borrar la memoria

Si, debe ser privilegiada, ya que por ahí se pueden borrar cosas importantes (como el mismo sistema operativo).

d) Deshabilitar las interrupciones

Si, debe ser privilegiada, ya que las interrupciones son atendidas por el sistema operativo y este debe decidir todo sobre ellas.

e) Cambiar a modo usuario

No necesariamente, ya que el modo usuario es el típico que se está en la computadora cuando se usa normalmente, entonces está bien poder estar en este modo.

f) Cambiar a modo monitor

Si

g) Leer una pista/sector de un disco magnético

Si, debería ser privilegiada ya que un malicioso podría acceder a datos sensibles.

24. ¿Es posible construir un S.O. seguro cuando el hardware no provee operación en modo dual?

El problema de no proveer operación en modo dual es que cualquier programa de usuario podría acceder a ciertas operaciones restringidas sin límites, ya que no habría manera de diferenciar si el que pretende acceder a esa operación es el SO, o una app cualquiera, por lo que un malicioso podría modificar el timer o algún I/O, o borrar archivos del sistema por

ejemplo. Este punto es el más crítico, ya que se podría borrar el sistema operativo sin mucho problema, por lo que no, sin modo dual no se puede asegurar que el SO sea seguro. Añadiendo a esto, la profe dijo que si bien se podría idealmente hacer un código microprogramado que tenga en cuenta todo, sería muy costoso.

25. Algunos sistemas de cómputo antiguos protegían al S.O. alojándose en una partición de memoria especial, donde no podía ser modificada por ninguna tarea de un usuario, ni tampoco por el mismo S.O. ¿Qué dificultades tiene este esquema?

Answer:

The data required by the operating system (passwords, access controls, accounting information, and so on) would have to be stored in or passed through unprotected memory and thus be accessible to unauthorized users.

Los sistemas operativos son software que se necesitan actualizar constantemente, como se ve hoy en día con Windows. Si el sistema operativo no puede accederse para actualizarse, entonces hay un problema, ya que si se encuentra un bug no se puede resolver, o al menos no tan fácilmente.

26. A continuación se detalla una lista de operaciones que normalmente son protegidas. Indique el conjunto mínimo que deben protegerse.

- a) leer del área de memoria del monitor
- b) escribir en el área de memoria del monitor
- c) obtener (fetch) una instrucción del área de memoria del monitor
- d) ejecutar un trap
- e) acceder al área de I/O (lectura o escritura).

Mínimamente, se deben proteger la b (para evitar que se modifiquen zonas sensibles) y la e (por la misma razón que la b) .

27. ¿Cuál es el propósito del intérprete de comandos? ¿Por que usualmente está separado del kernel?

El propósito de intérprete es el de ser una herramienta para buscar y ejecutar comandos ingresados por el usuario, ya sea directamente, o mediante llamadas al sistema (system call). Está separado ya que puede ser objeto de cambios constantes, entonces al estar separado es más fácil realizar estos cambios.

28. ¿Cual es el propósito de los systems calls?

El de proveer una interfaz para los servicios que provee el SO.

29. ¿Cuál es la principal ventaja de usar un microkernel en el diseño de sistemas? ¿Cómo interactúan los programas de usuario y los servicios del sistema en una arquitectura basada en microkernel? ¿Cuáles son las desventajas de usar la arquitectura de microkernel?

La ventaja es que, al tener las funcionalidades modularizadas por fuera del kernel, es mucho más fácil reemplazar, modificar y expandir el SO, lo que resulta en que puede ser portado con mayor facilidad. Además, aporta mayor seguridad y confiabilidad, ya que si, por ejemplo, llega a fallar un módulo (el de audio por ejemplo), no se cae todo el sistema.

En una arquitectura de microkernel, al estar las aplicaciones de usuario y los programas separados del kernel, estos se comunican entre sí a través de mensajes, los cuales se intercambian a través del kernel.

La principal desventaja es que, al estar todo modularizado y comunicarse estos módulos a través del kernel, estos canales de comunicación se pueden ver saturados, lo que hace que el rendimiento se vea afectado. Un dato extra es que una de las primeras versiones de Windows habían separado tanto las funcionalidades, que tuvieron que cambiar todo ya que el sistema se volvía demasiado lento como para ser usado.

30. ¿En que se asemejan la arquitectura kernel modular y la arquitectura en niveles? ¿En que se diferencian?

Ambas arquitecturas se asemejan al hecho de tener ciertas funciones separadas en módulos (en el caso de la de niveles, en capas, y en microkernel, en distintos módulos). Esto da la ventaja a ambas arquitecturas de poder ser implementadas y cambiadas con relativa facilidad (al menos en comparación a la monolítica). Además, ambas tienen en común algo malo, y es que el rendimiento en ambas es pobre en comparación a la monolítica. En el caso de la de capas, puede ser que haya que atravesar muchas capas para comunicar algo de una capa a otra, y lo del microkernel ya se mencionó arriba.

Y justamente ahí se encuentra la principal diferencia, y es que en la de capas, una capa solo puede solicitar servicios a cualquiera de las capas que se encuentren debajo de ella, mientras que en microkernel cualquiera de los módulos se pueden comunicar entre sí.

31. ¿Cual es la ventaja para un diseñador de sistemas operativos la utilización de una máquina virtual? ¿y para un usuario?

Podría servir para ejecutar updates, modificar y realizar acciones riesgosas sin afectar al hardware de la computadora (ya que está aislado por la virtualización).

Para un usuario, podría servir para probar distintos SO para ver si es el adecuado, sin ningún riesgo para el sistema, y sin reemplazar el SO actual. En particular, para un estudiante, podría servir para revisar las minucias y los procesos que ocurren detrás de un SO, sin riesgo. Además, serviría para probar distintas aplicaciones (algunas solo disponibles para x sistemas operativos), y ver si pueden correr en ese SO.